

PERSONAL EXPENSE TRACKER

PROJECT REPORT

Name : Piyush Dhanraj
Reg No. : 26BCE11079
Course : B.Tech CSE (Core)
Subject : Introduction to Problem Solving and Programming
Semester : 1st Semester
Academic Year : 2026-27

1. Introduction

Keeping track of daily expenses can become difficult when expenses are recorded in different places or are simply remembered mentally. I made this Personal Expense Tracker as a simple Python program to keep expense records in one place.

The program runs in the terminal and uses a CSV file to store the expense records. A user can enter the date, description, category and amount of an expense. The program also provides options to view expenses, calculate the total amount, delete an expense and see a category-wise summary.

2. Problem Statement

Managing small day-to-day expenses manually can make it difficult to know how much money has been spent and where it has been spent. There is also a chance of losing or forgetting individual expense records.

The purpose of this project is to provide a simple program where expense details can be entered, stored and viewed whenever required. The program also helps the user calculate total spending and check spending according to categories.

3. Objectives

- To create a simple Python program for recording personal expenses.
- To store expense records in a CSV file so that they can be loaded again.
- To allow the user to view all saved expenses.
- To calculate the total amount spent.
- To allow an existing expense to be deleted.
- To display a category-wise summary of expenses.
- To practise Python concepts such as lists, dictionaries, loops, file handling and exception handling.

4. Functional Requirements

The main functions provided by the project are:

- Add Expense – The user can enter the date, description, category and amount.
- View Expenses – The program displays all currently stored expenses.
- Calculate Total – The program adds the amounts of all expenses and displays the total.
- Delete Expense – The user can select an expense number and the selected record will be removed.

- Category Summary – The program adds expenses belonging to the same category and displays the category wise total.
- CSV Storage – Expense records are stored in csv file so that they can be loaded when the program is started again.

5. Non-Functional Requirements

- Usability – The program uses a simple numbered menu so that the user can select an operation easily.
- Reliability – Expense data is saved in a CSV file and is loaded when the program starts.
- Error Handling – Invalid amount input is handled using try/except, and negative amounts are not accepted.
- Maintainability – The project has a clear folder structure with the main code, data file, README file, statement file and a test file in which basic amount validation tests were performed.

6. System Architecture

The project follows a simple console based structure. The user interacts with the program through the terminal menu. The main Python program processes the selected option and works with the expense list. The CSV file is used for permanent storage of the expense records.

Basic flow:

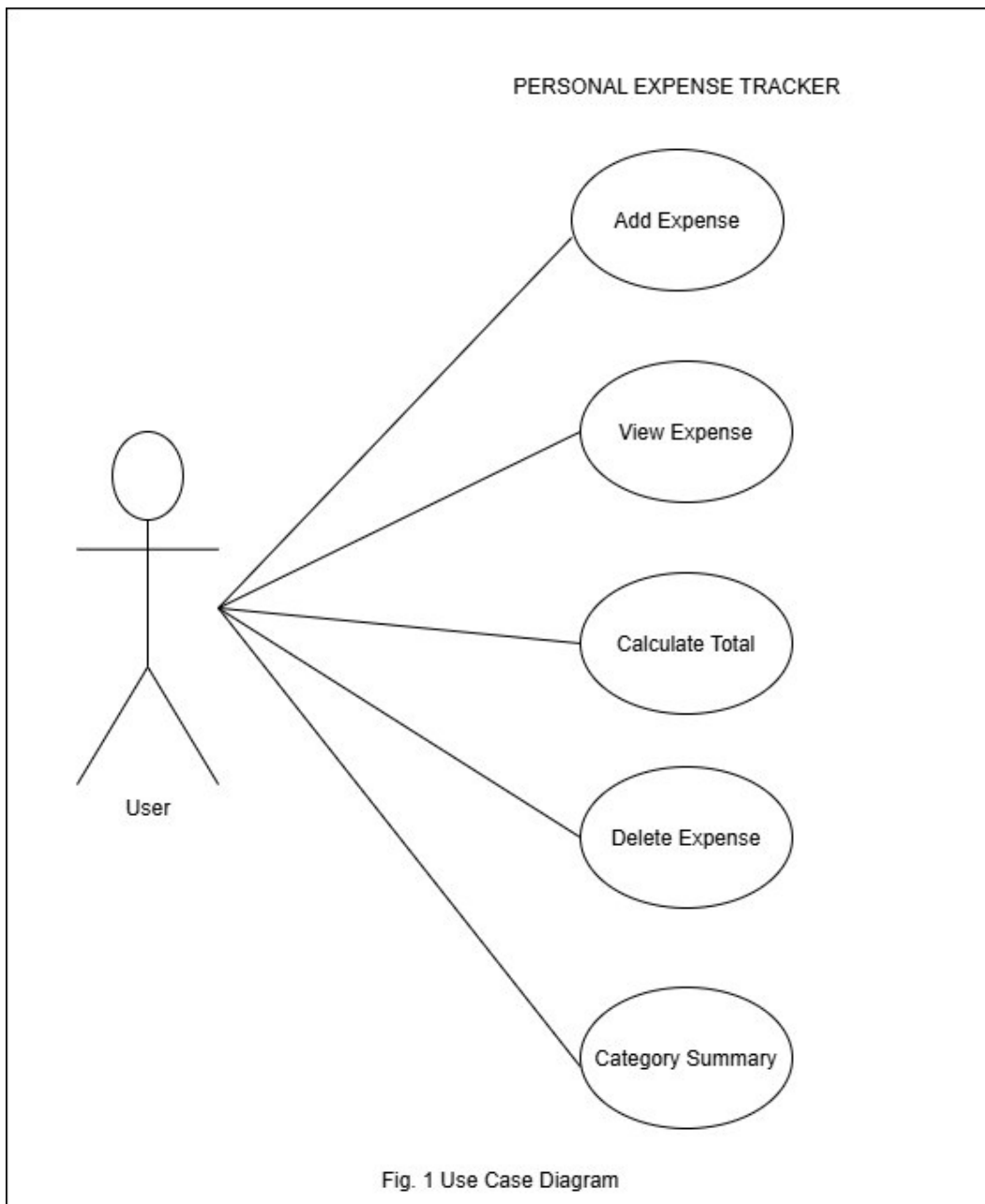
User → Main Python Program → Expense List → CSV File

When the program starts, it reads the existing CSV file and stores the records in a list of dictionaries. When a new expense is added, it is added to the list and also written to the CSV file. When an expense is deleted, the CSV file is rewritten with the remaining records.

7. Design Diagrams

The following diagrams show how the system works:

- Use Case Diagram – shows the user and the main operations of the Expense Tracker.
- Workflow Diagram – shows the flow from opening the program to selecting an option and completing an operation.
- Sequence Diagram – shows the interaction between the user, Python program and CSV storage.
- Component Diagram – shows the main program and the data structure used for expenses.
- ER/Storage Diagram – shows the fields stored for each expense in the CSV file.



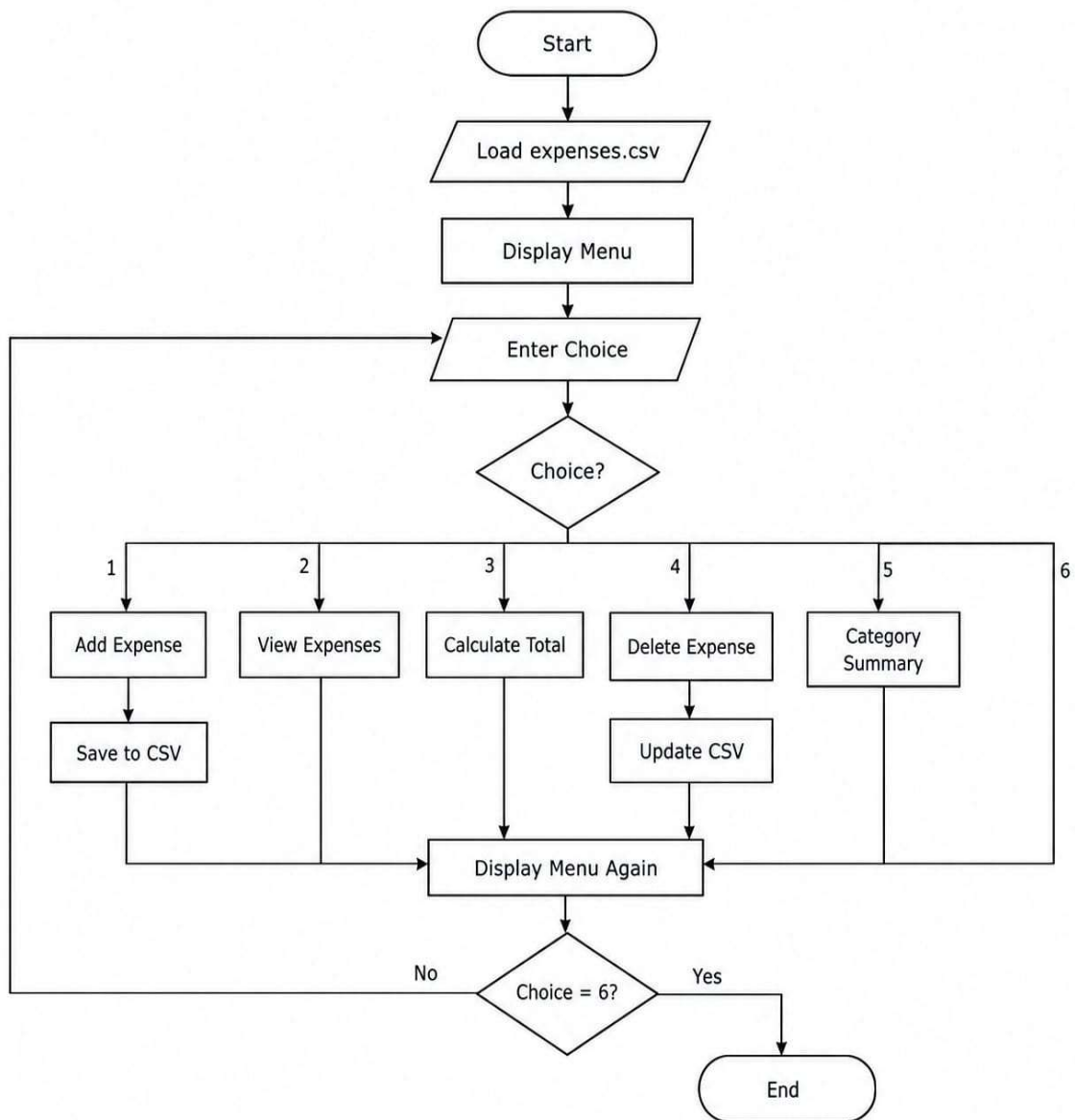
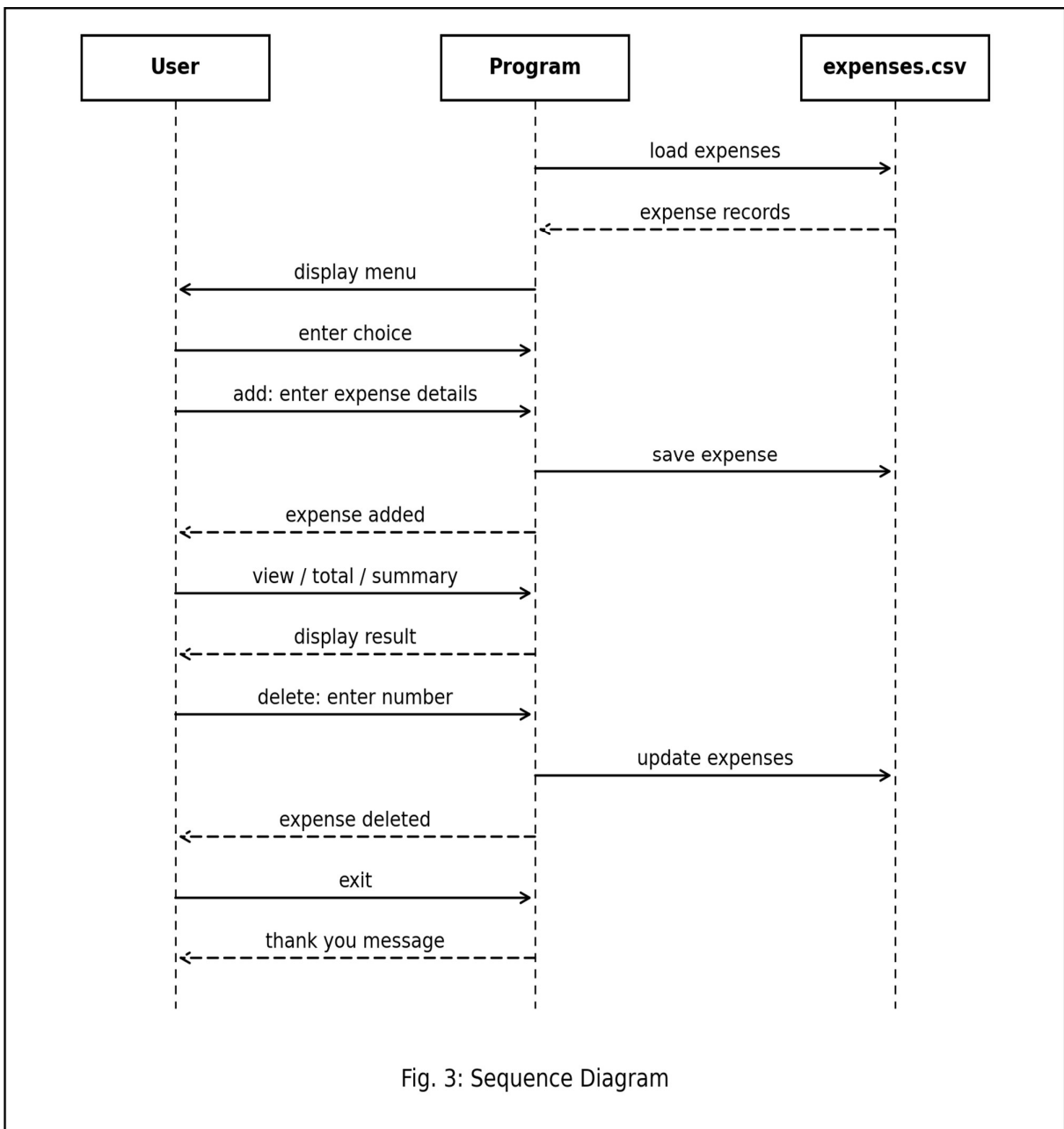
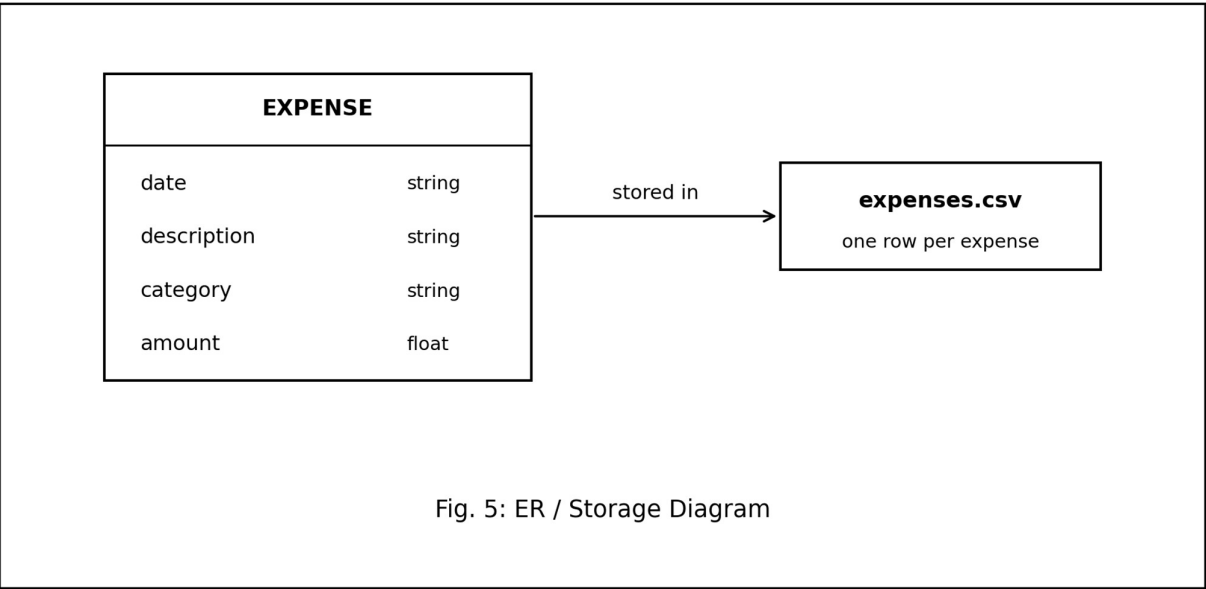
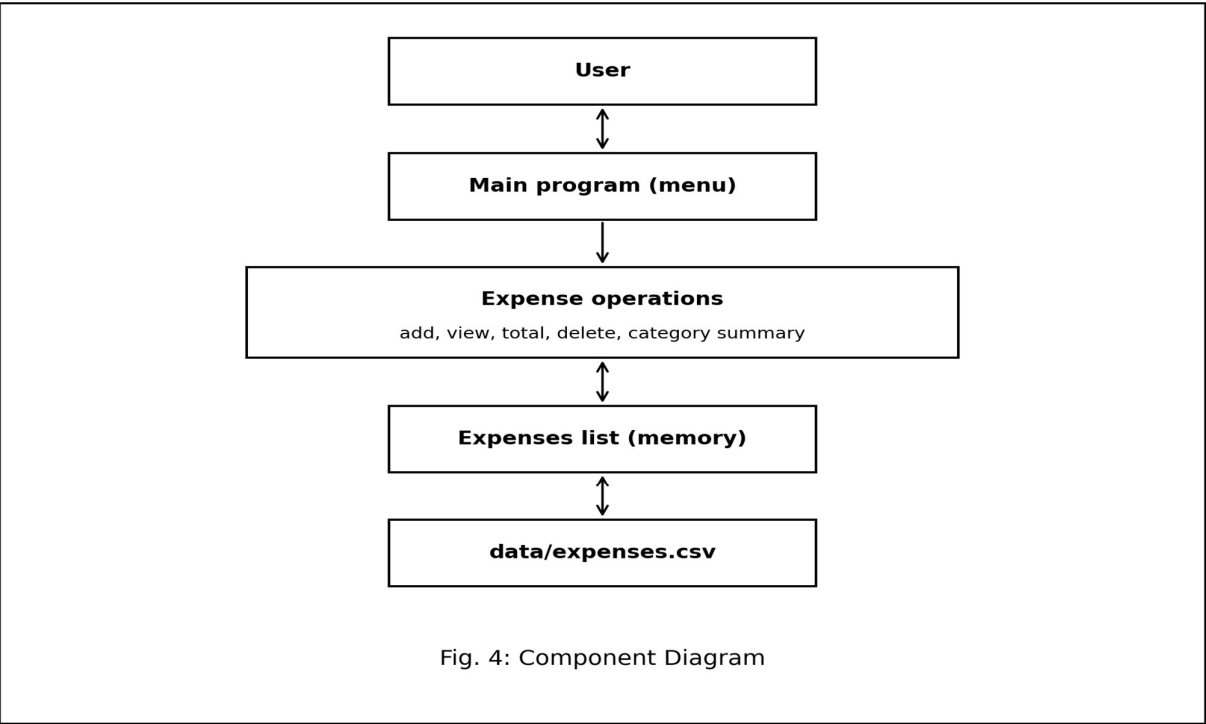


Fig. 2: Workflow Diagram





8. Design Decisions and Rationale

I used a list to store multiple expense records while the program is running. Each expense is stored as a dictionary because it makes the individual fields such as date, description, category and amount easy to access.

I used a CSV file instead of a database because the project is small and the data is simple. CSV file handling also gives practical experience with reading and writing data to a file.

The program uses a menu-driven approach because it keeps the interaction simple for a beginner-level console application.

9. Implementation Details

The project is written in Python. The csv module is used to read and write expense records.

At the beginning of the program, the existing CSV file is opened and its records are converted into dictionaries. If the file does not exist, the program continues without showing an error because the project may be running for the first time.

For adding an expense, the program takes the required details from the user. The amount is converted to a floating-point number. A while loop and try/except are used so that invalid numeric input can be handled.

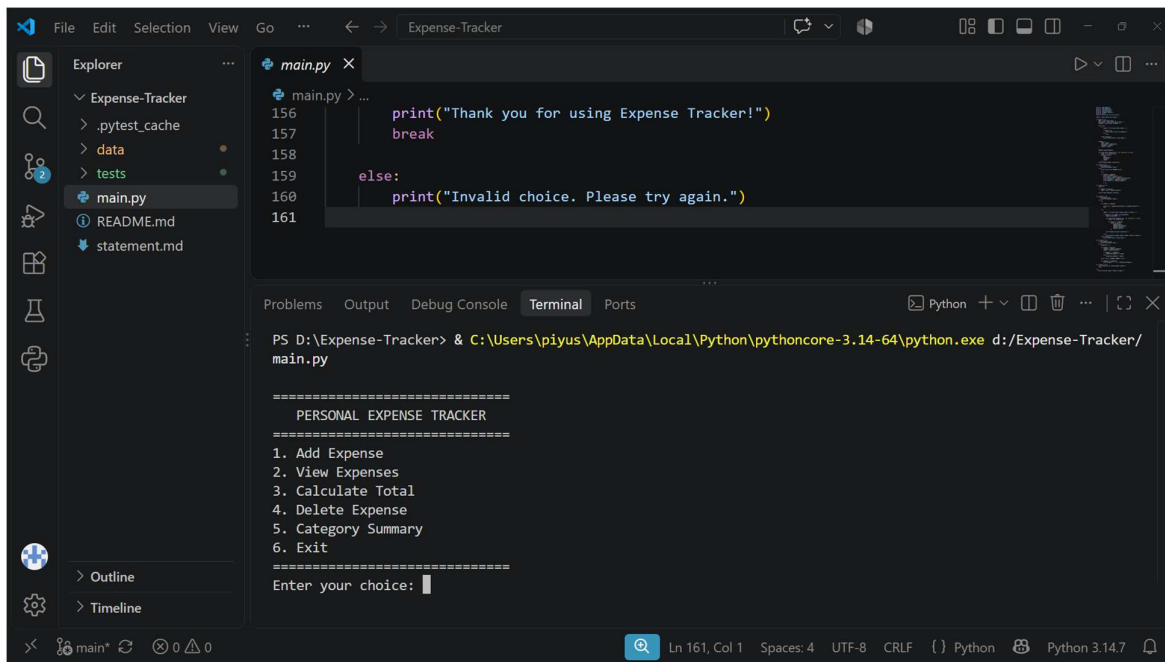
For deleting an expense, the program displays the available expenses with numbers. After the user selects a number, the corresponding item is removed from the list and the CSV file is rewritten.

For the category summary, a dictionary is used to keep the total amount for each category. If a category is already present, its amount is increased; otherwise, a new category entry is created.

10. Screenshots / Results

The program was run successfully from the VS Code terminal. The following screenshots are added to show the working of the project:

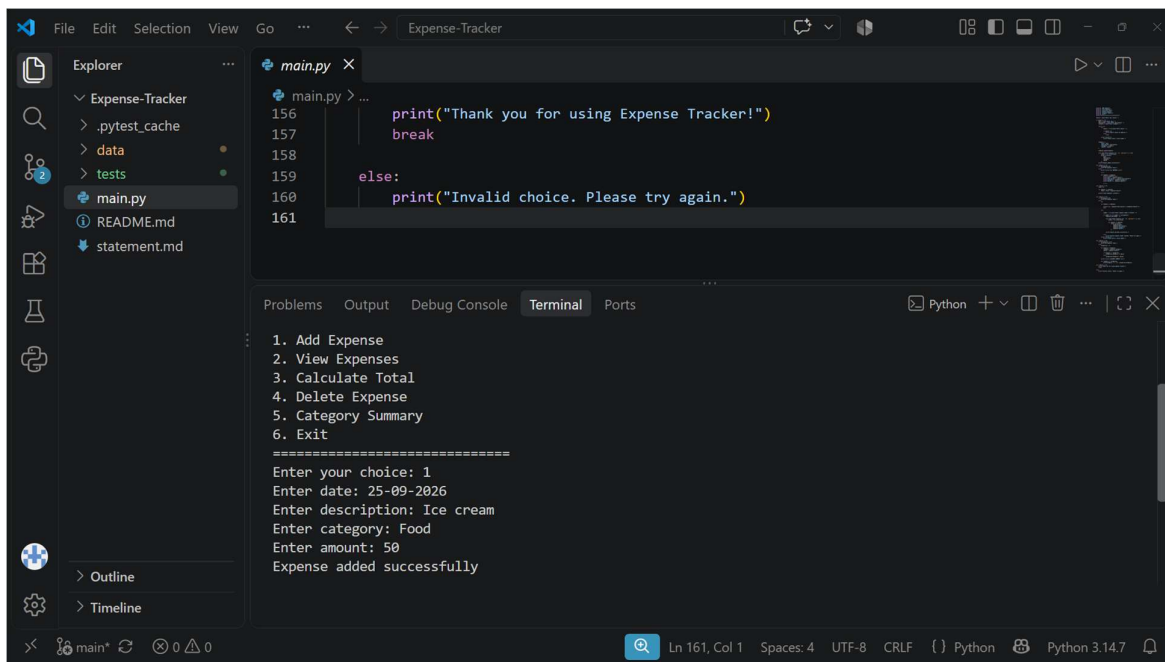
- Main menu
- Adding an expense
- Viewing expenses
- Calculating total expenses
- Deleting an expense
- Category summary
- GitHub repository



```
main.py
156     print("Thank you for using Expense Tracker!")
157     break
158
159     else:
160         print("Invalid choice. Please try again.")
161
```

PS D:\Expense-Tracker> & C:\Users\piyus\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/Expense-Tracker/main.py

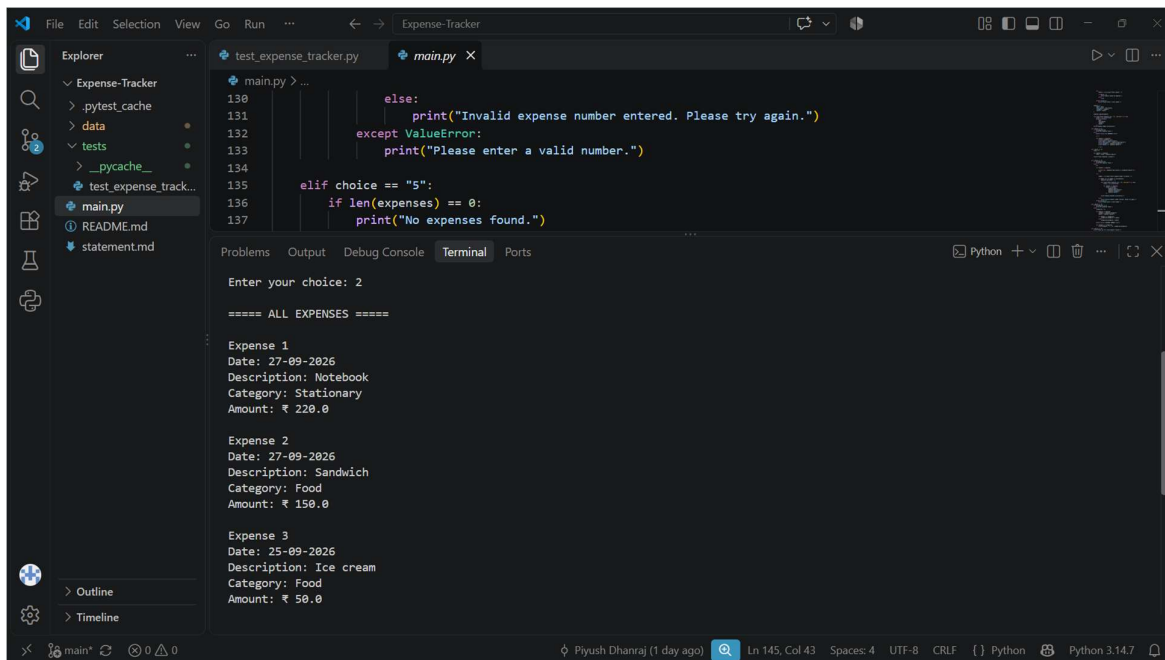
```
=====
PERSONAL EXPENSE TRACKER
=====
1. Add Expense
2. View Expenses
3. Calculate Total
4. Delete Expense
5. Category Summary
6. Exit
=====
Enter your choice: 
```



```
main.py
156     print("Thank you for using Expense Tracker!")
157     break
158
159     else:
160         print("Invalid choice. Please try again.")
161
```

1. Add Expense
2. View Expenses
3. Calculate Total
4. Delete Expense
5. Category Summary
6. Exit

Enter your choice: 1
Enter date: 25-09-2026
Enter description: Ice cream
Enter category: Food
Enter amount: 50
Expense added successfully



```
File Edit Selection View Go Run ... Expense-Tracker
```

Explorer

- Expense-Tracker
 - .pytest_cache
 - data
 - tests
 - __pycache__
 - test_expense_track...
 - main.py
 - README.md
 - statement.md

main.py

```
130         else:
131             print("Invalid expense number entered. Please try again.")
132     except ValueError:
133         print("Please enter a valid number.")
134
135     elif choice == "5":
136         if len(expenses) == 0:
137             print("No expenses found.")
```

Problems Output Debug Console Terminal Ports

Enter your choice: 2

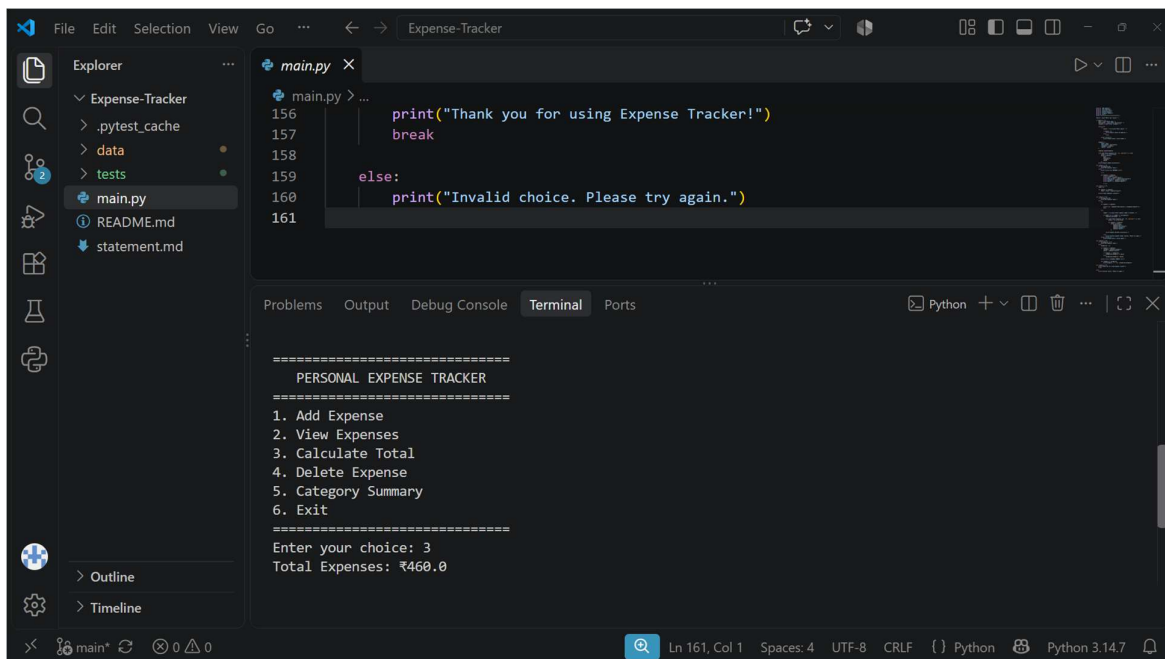
===== ALL EXPENSES =====

Expense 1
Date: 27-09-2026
Description: Notebook
Category: Stationary
Amount: ₹ 220.0

Expense 2
Date: 27-09-2026
Description: Sandwich
Category: Food
Amount: ₹ 150.0

Expense 3
Date: 25-09-2026
Description: Ice cream
Category: Food
Amount: ₹ 50.0

main* 0 0 0 Piyush Dhanraj (1 day ago) Ln 145, Col 43 Spaces: 4 UTF-8 CRLF {} Python Python 3.14.7



```
File Edit Selection View Go ... Expense-Tracker
```

Explorer

- Expense-Tracker
 - .pytest_cache
 - data
 - tests
 - main.py
 - README.md
 - statement.md

main.py

```
156         print("Thank you for using Expense Tracker!")
157         break
158
159     else:
160         print("Invalid choice. Please try again.")
161
```

Problems Output Debug Console Terminal Ports

PERSONAL EXPENSE TRACKER

1. Add Expense
2. View Expenses
3. Calculate Total
4. Delete Expense
5. Category Summary
6. Exit

Enter your choice: 3
Total Expenses: ₹460.0

main* 0 0 0 Ln 161, Col 1 Spaces: 4 UTF-8 CRLF {} Python Python 3.14.7

```
main.py
156     print("Thank you for using Expense Tracker!")
157     break
158
159     else:
160         print("Invalid choice. Please try again.")
161
```

3. Calculate Total
4. Delete Expense
5. Category Summary
6. Exit

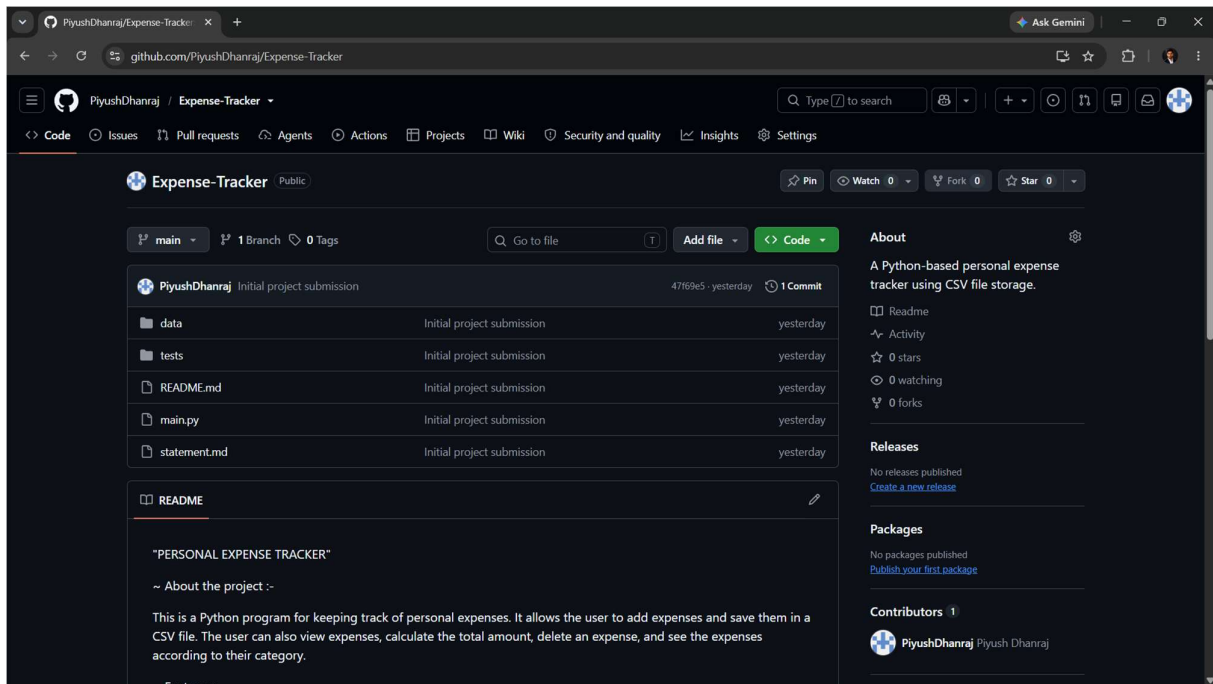
Enter your choice: 4
1. Notebook ₹220.0
2. Pens ₹40.0
3. Sandwich ₹150.0
4. Ice cream ₹50.0
Enter expense number to delete: 2
Expense deleted successfully.

```
main.py
156     print("Thank you for using Expense Tracker!")
157     break
158
159     else:
160         print("Invalid choice. Please try again.")
161
```

1. Add Expense
2. View Expenses
3. Calculate Total
4. Delete Expense
5. Category Summary
6. Exit

Enter your choice: 5

===== CATEGORY SUMMARY =====
Stationary : ₹ 220.0
Food : ₹ 200.0
=====



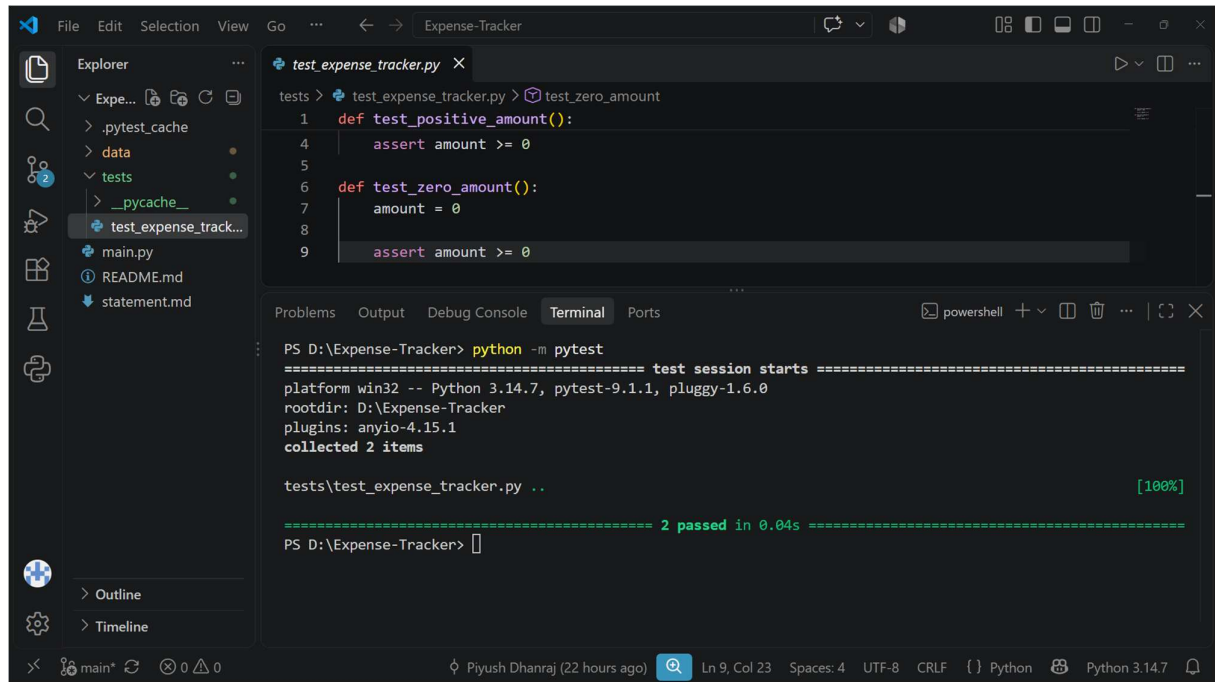
<https://github.com/PiyushDhanraj/Expense-Tracker>

11. Testing Approach

I created a small test file using pytest to check the basic amount rule used in the project, which is that an amount must not be negative. Two simple tests were written using assert statements. The first checks that a positive amount such as 500 is accepted, and the second checks that an amount of 0 is accepted.

The program is not divided into functions, so these tests do not call the program code directly they only check the rule itself.

The tests were run using the command: `python -m pytest`. The result was 2 passed.



The screenshot shows a Visual Studio Code editor window with the file `test_expense_tracker.py` open. The file contains two test functions: `test_positive_amount()` and `test_zero_amount()`, both using `assert` statements to verify that the amount is non-negative. The terminal at the bottom shows the command `python -m pytest` being executed, resulting in two tests passing successfully. The terminal output includes the pytest version (9.1.1), the root directory (D:\Expense-Tracker), and the test results for `tests\test_expense_tracker.py`.

```
tests > test_expense_tracker.py > test_zero_amount
1 def test_positive_amount():
4     assert amount >= 0
5
6 def test_zero_amount():
7     amount = 0
8
9     assert amount >= 0

PS D:\Expense-Tracker> python -m pytest
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\Expense-Tracker
plugins: anyio-4.15.1
collected 2 items

tests\test_expense_tracker.py .. [100%]

===== 2 passed in 0.04s =====
PS D:\Expense-Tracker>
```

12. Challenges Faced

One part that required attention was handling the expense data correctly when the CSV file was moved into the data folder. The file paths in the Python program had to be changed accordingly.

Another part was handling invalid amount input. The program needed to avoid crashing when the user entered something that could not be converted into a number. This was handled using `try/except` and a loop.

I also had to set up Git and GitHub for version control and make sure that the project files were pushed to the correct repository.

13. Learnings and Key Takeaways

While making this project, I got more practice with basic Python programming and learned how different concepts can be combined in a complete application.

- I learned how to read and write CSV files using Python.
- I practised storing related information in dictionaries and multiple records in a list.
- I understood how loops can be used for menus and repeated input validation.
- I used exception handling to deal with invalid user input.
- I learned the basic process of using Git and GitHub to store and manage a project.

14. Future Enhancements

- Add an option to edit an existing expense.
- Add monthly and yearly expense reports.
- Add charts to make spending patterns easier to understand.
- Add a search or filter option for expenses.
- Add a graphical or web-based interface in a future version.
- Use a database such as SQLite if the amount of stored data becomes larger.

15. References

- Python documentation and concepts used during development.
- Pytest documentation for basic testing.
- Git and GitHub documentation for version control.